Problema 1.11. Serie de Fibonacci

Desarrolla un programa que lea un entero n≥0 y muestre los primeros n términos de la serie de Fibonacci (0, 1, 1, 2, 3, 5, 8, ...), donde cada término es la suma de los dos anteriores.

1. Análisis del problema

- Entrada: n (número entero).
- Proceso: empezar con anterior = 0 y actual = 1; repetir n veces. En cada vuelta se muestra el primero, se suman los dos para obtener el siguiente y se avanza un lugar en la serie.
- Salida: en pantalla aparecen los n primeros números de la serie.

Restricción: n tiene que ser 1 o más.

2. Tabla de objetos

Objeto	Nombre	valor	Tipo
Entrada	n	variable	Entero
control	i	variable	Entero
Trabajo	anterior	variable	Entero
Trabajo	actual	variable	Entero
Trabajo	siguiente	variable	Entero
Salida	serie	var(pantalla)	Pantalla

3. Seudocódigo

```
Algoritmo SerieFibonacci
    Definir n, anterior, actual, siguiente Como Entero
    Escribir "Ingresar el valor de n: ";
    Leer n

    Si n≥1 Entonces
        actual = 1;
        anterior = 0;

        Para i=0 hasta n-1 Hacer
            Escribir anterior;
            siguiente = anterior + actual;
            anterior = actual;
```

Escaneado con CamScanner

actual = siguiente;

FinPara

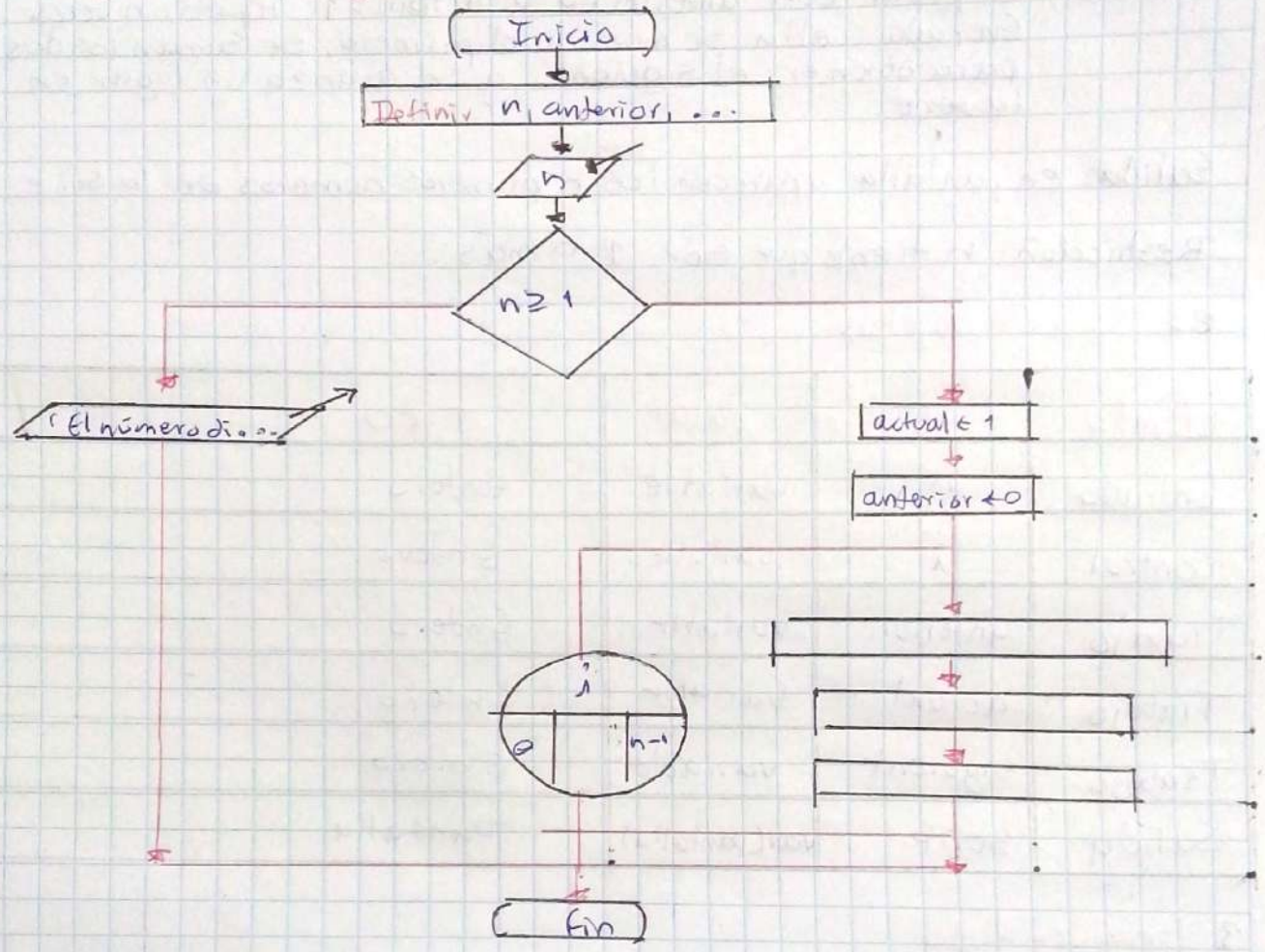
SiNo

Escribir "El número digitado debe ser mayor o igual a 1";

FinSi

FinAlgoritmo

4. Diagrama de Flujo



5. Prueba de escritorio

Etapa	Descripción	i	anterior	actual	siguiente	Pantalla
Entrada	Escribe n	-	-	-	-	n = 4
Proceso	Valores iniciales	-	0	1	-	-
	Vuelta 1: 0+1 (siguiente)	0	1	1	1	0
	Vuelta 2: 1+1	1	1	2	2	0 1

Escaneado con CamScanner

	Vuelta 3: siguiente = 1+2	2	2	3	3	0 1 1
	Vuelta 4: siguiente = 2+3					0 1 1 2
Salida	fin del ciclo					Serie: 0 1 1 2

Justificación: ¿Cómo cambia tu algoritmo si solo debes mostrar el término número n y no toda la serie?

El algoritmo casi no cambia, se sigue calculando la serie con las mismas variables (anterior, actual y siguiente), pero dentro del ciclo ya no se muestra nada. Se repite el proceso n-1 veces y, al terminar, se muestra solo el valor de anterior, que es el término número n.

Problema 1.12. Suma de dígitos

Desarrolla un programa que lea un entero positivo y calcule la suma de sus dígitos.

1. Análisis del problema

Entrada: el progra pide un número entero

Proceso: en cada vuelta se toma el último dígito del número (n % 10), se suma a un acumulador y luego se quita ese dígito dividiendo el número entre 10. Esto se repite hasta que el número llegue a 0.

Salida: en pantalla aparece la suma de los dígitos.

Restricción: n debe ser mayor a 0.

2. Tabla de objetos

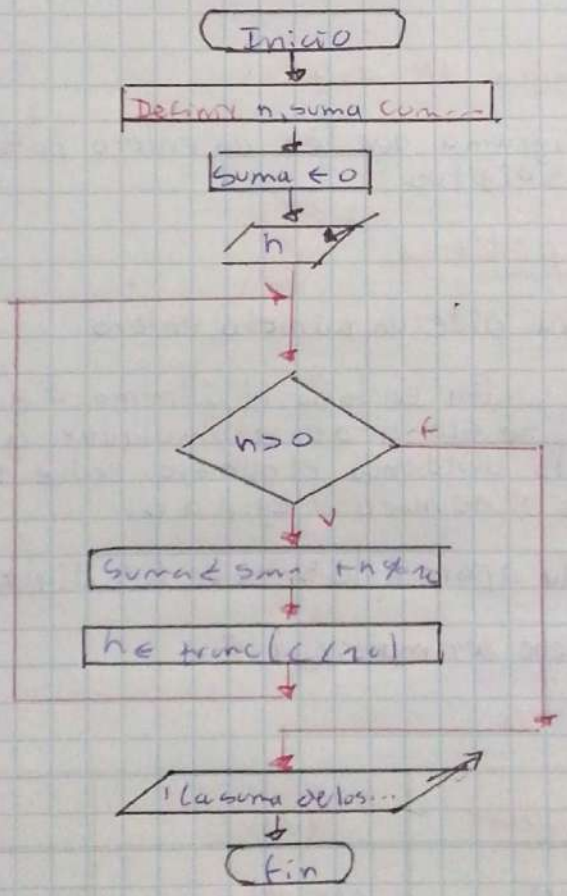
Objeto	Nombre	Valor	Tipo
Dato de entrada	n	variable	Entero
trabajo	Suma	variable	Entero
trabajo			
Dato de salida	Suma	variable (pantalla)	Entero

Escaneado con CamScanner

3. Seudocódigo

```
Algoritmo SumaDigitos
    Definir n, suma Como Entero
    suma ← 0;
    Escribir "Ingrese un número positivo: ";
    Leer n;
    Mientras (n>0) Hacer
        suma ← suma + n%10;
        n ← trunc(n/10);
    FinMientras
    Escribir "La suma de los digitos es: ", suma;
FinAlgoritmo
```

4. Algoritmo



Escaneado con CamScanner

5. Prueba de escritorio

Etapa	Descripción	n	suma	Pantalla
Entrada	Ingrese un número: 452	452	–	–
Proceso	Valor inicial	452	0	–
	vuelta 1: suma = 0 + 452%10 (2) n = 452 / 10	45	2	–
	vuelta 2: suma = 2 + 45%10 = 5 n = 45 / 10	4	7	–
	vuelta 3: suma = 7 + 4%10 = 4 n = 4/10	0	11	–
Salida	n llegó a 0, termina el ciclo y se muestra la suma.	0	11	La suma de los dígitos es: 11

Justificación: ¿cuántas veces se repite el ciclo para un número de k dígitos?

En cada vuelta se elimina un solo dígito del número (el último), porque dividir entre 10 y quedarse con la parte entera le quita una cifra. Un número K dígitos necesita K divisores para llegar a 0, y en ese momento la condición n>0 deja de cumplirse y el ciclo termina.

Problema 1.13. Número capicúa

Desarrolla un programa que lea un entero positivo y determine si es capicúa, es decir, si se lee igual de izquierda a derecha que de derecha a izquierda.

1. Análisis del problema

- Entrada: el usuario escribe un número positivo.
- Proceso: se guarda una copia de número original. Luego, en cada vuelta se toma el último dígito, se agrega al final del número invertido y se quita ese dígito de n. Cuando n llega a 0, se compara el invertido con el original.

Escaneado con CamScanner

Salida: en pantalla sale si es capicúa ono.

Restricción: n debe ser un entero positivo.

2. Tabla de objetos

Objeto	Nombre	Valor	Tipo
Dato de entrada	n	Variable	Entero
trabajo	original	Variable	Entero
trabajo	digito	Variable	Entero
trabajo	invertido	Variable	Entero
Dato de salida	Mensaje	Variable (pantalla)	texto

3. Seudocódigo

```
Algoritmo Capicua
    Definir n, original, digito, invertido Como Entero
    invertido ← 0.
    Escribir "Ingrese un número positivo: ";
    Leer n

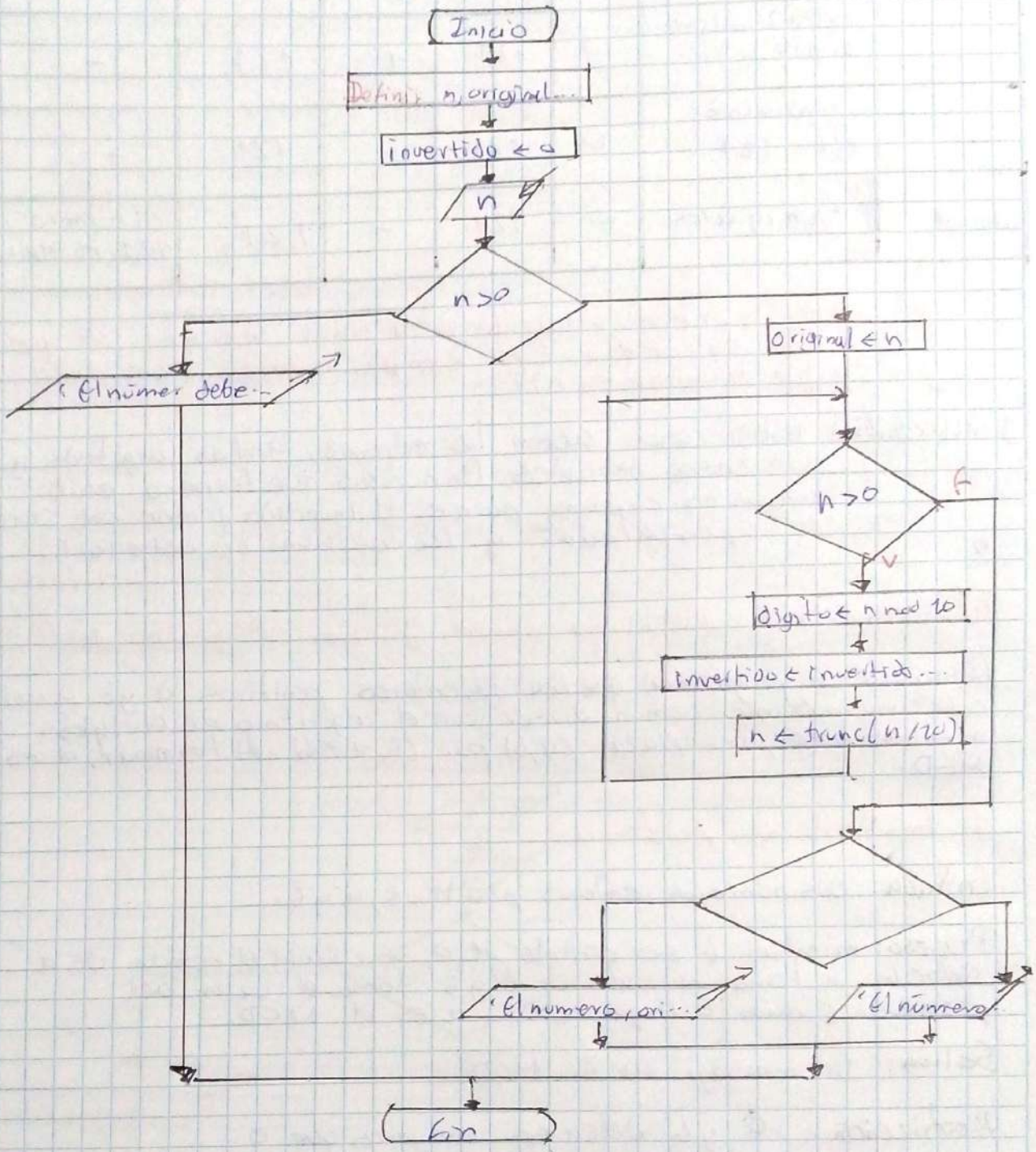
    Si n>0 Entonces:
        original ← n;

        Mientras n>0 Hacer
            digito ← n mod 10
            invertido ← invertido*10+digito;
            n ← trunc ( n/10 );
        FinMientras

        Si original = invertido Entonces
            Escribir " El número es capicúa ", original;
        SiNO
            Escribir " El número no es capicúa ", original;
        FinSi
    SiNO
        Escribir " El número debe ser positivo ";
    FinSi
FinAlgoritmo
```

Escaneado con CamScanner

4. Algoritmo



5. Prueba de escritorio

Etapa	Descripción	n	original	digito	invertido	Pantalla
Entrada	El usuario escribe: 121	121	–	–	–	–
Proceso	valores iniciales	121	121	–	0	–
	Vuelta 1: último dígito	12	121	1	1	–

Escaneado con CamScanner

	Vuelta 2: último dígito 2	1	121	2	12	-
	Vuelta 3: último dígito 1	0	121	1	121	-
	Comparación: 121 = 121	0	121	-	121	-
Salida	Son iguales	0	121	-	121	El número 121 es capicúa

¿Qué casos de prueba elegirías para estar seguro/a de que tu solución es correcta (por ejemplo, números de un solo dígito o que terminan en 0)?

Justificación: estos casos cubren los extremos, ambas longitudes y los casos delicados. Un número que termina en 0 nunca es capicúa, porque el invertido pierde ese cero y la comparación y la condición lo detecta.

Problema 1.14. Máximo común divisor (Algoritmo de Euclides)

Desarrolla un programa que lea dos enteros positivos a y b y calcule su máximo común divisor con el algoritmo de Euclides: mientras b ≠ 0, reemplaza (a, b) por (b, a mod b). Al terminar, a es MCD.

1. Análisis del problema

Entrada: dos números enteros positivos, a y b.

Proceso: mientras b sea distinto de 0, se calcula el resido de a entre b, a toma el valor de b y b toma el valor del residuo. Cuando b llega a 0, a es el MCD.

Salida: Un mensaje con el MCD.

Restricción: a y b deben ser mayores que 0.

2. Tabla de objetos

Objeto	Nombre	Valor	Tipo
Dato de entrada	a	Variable	Entero
Dato de entrada	b	Variable	Entero
Trabajo	residuo	Variable	Entero
Dato de salida	MCD(a)	Variable(pantalla)	Entero

Escaneado con CamScanner

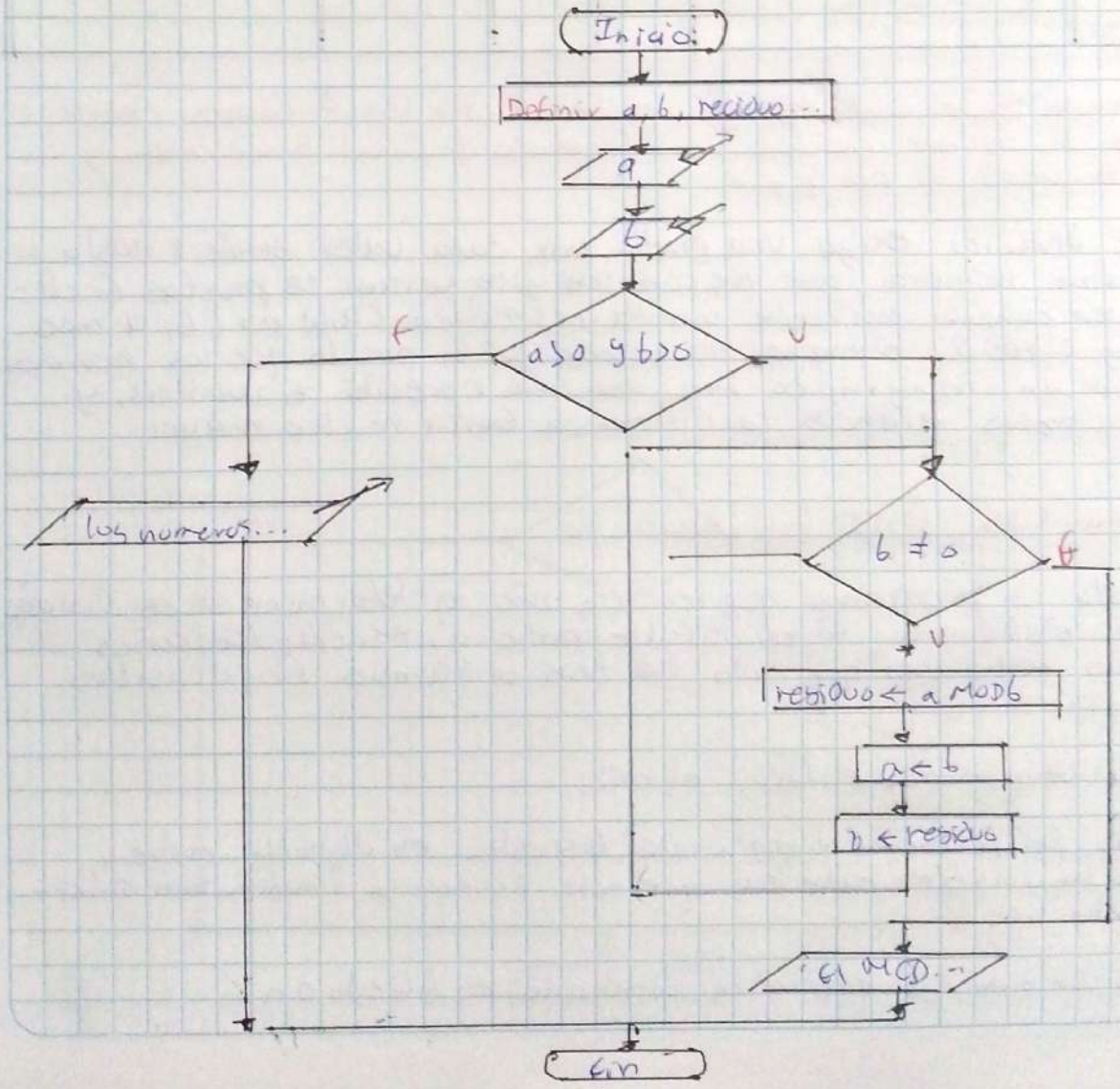
3. Seudocódigo

```
Algoritmo MCD
    Definir a, b, residuo Como Entero
    Escribir "Ingrese el valor de a: ";
    Leer a;
    Escribir "Ingrese el valor de b: ";
    Leer b;

    Si a>0 y b>0 Entonces
        Mientras b ≠ 0 Hacer
            residuo ← a mod b;
            a ← b;
            b ← residuo;
        FinMientras

        Escribir "El MCD es: ", a;
    SiNo
        Escribir "Los números deben ser positivos";
    FinSi
FinAlgoritmo
```

4. Algoritmo



Escaneado con CamScanner

5. Prueba de escritorio

Etapa	Descripción	a	b	residuo	Pantalla
Entrada	El usuario escribe 48 y 18	48	18	—	—
Proceso	Valores iniciales	48	18	—	—
	Vuelta 1; residuo= 48 % 18 = 12 a = 18 ; b = 12	18	12	12	—
	Vuelta 2; residuo = 18 % 12 = 6 a = 12 ; b = 6	12	6	6	—
	Vuelta 3, residuo = 12 % 6 = 0 a = 6 ; b = 0	6	0	0	—
Salida	b = 0, termina el ciclo	0	0	0	El MCD es: 6

Compara este método con probar todos los divisores desde 1 hasta el menor de los dos números. ¿Cuál hace menos operaciones y por qué?

Probar divisores exige una prueba por cada valor desde 1 hasta el menor número, así que con 48 y 18 serían 18 pruebas. Euclides, en cambio, en cada vuelta reemplaza (a, b) por (b, a mod b) y el residuo siempre es menor que b, por lo que los números se achican muy rápido. Por eso solo necesité 3 vueltas, y con números grandes la diferencia sería mucho mayor.

Problema 1.15 Año bisiesto

Desarrolla un programa que lea un año y determine si es bisiesto. Un año es bisiesto si es divisible entre 4, excepto los años divisibles entre 100, que solo lo son si también son divisibles entre 400.

Entrada: un número entero, el año.

Proceso: Se revisa la regla del bisiesto, es divisible entre 4, excepto los divisibles entre 100, que solo lo son si también son divisibles entre 400.

Salida: un mensaje que indica si el año es bisiesto o no.

Escaneado con CamScanner

Restricción: el año debe ser mayor a 0.

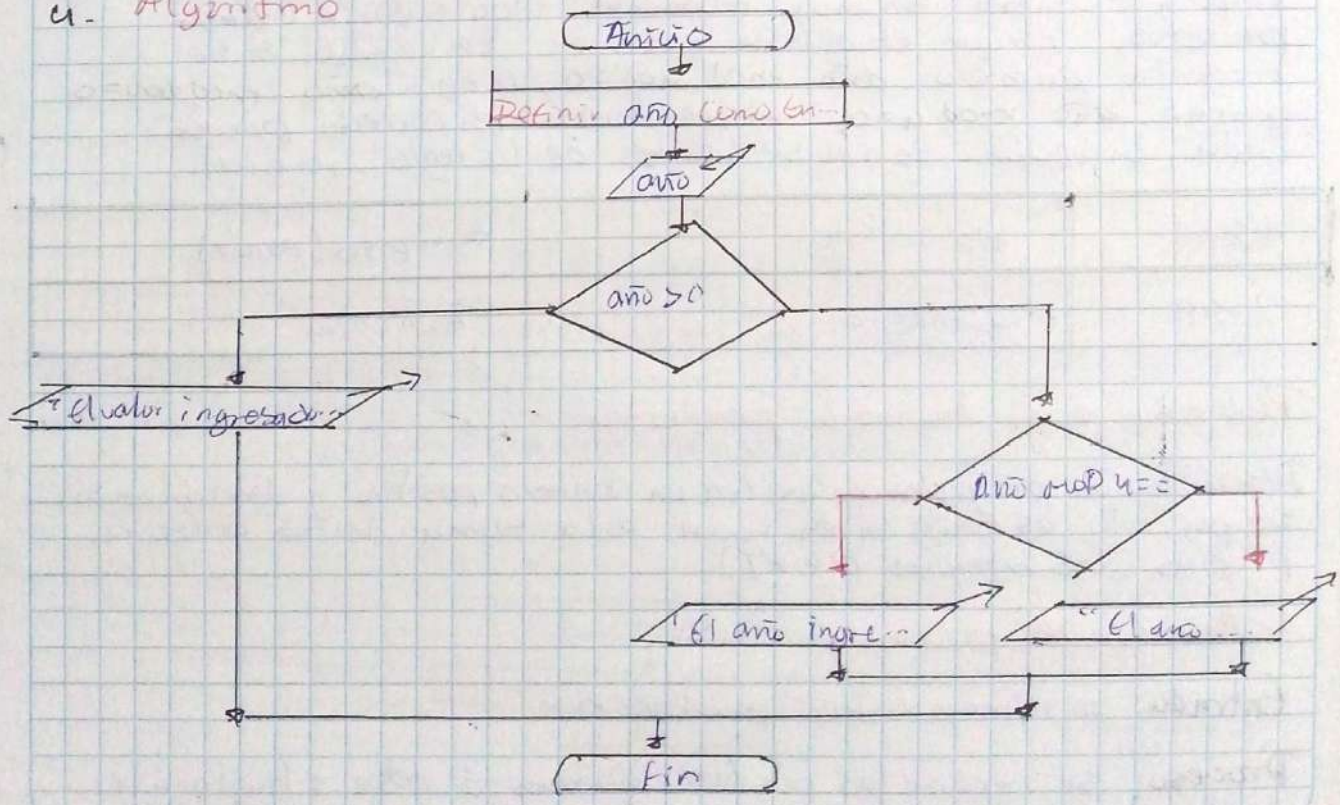
2. Tabla de objetos

Objeto	Nombre	Valor	Tipo
Dato de entrada	año	variable	Entero
Dato de salida	Mensaje	Variable (Pantalla)	Texto

3. Seudocodigo

```
Algoritmo AñoBisiesto
    Definir año Como Entero
    Escribir "Ingrese el año:";
    Leer año;
    Si (año > 0 ) Entonces
        Si (año mod 4 == 0 y año mod 100 <> 0) O (año % 400 == 0)
        Entonces
            Escribir "El año ingresado es bisiesto:", año;
        SiNo
            Escribir "El año ingresado no es bisiesto.", año;
        FinSi
    SiNo
        Escribir "El valor ingresado debe ser mayor a 0";
    FinSi
FinAlgoritmo
```

4. Algoritmo



Escaneado con CamScanner

5. Prueba de escritorio

Etapa	Descripción	año	Pantalla
Entrada	El usuario escribe 1900	1900	—
Proceso	1900 mod 4 = 4 → verdadero	1900	—
	1900 mod 100 ≠ 0 → falso	1900	—
	Primera parte: verdadero y falso = falso	1900	—
	1900 mod 400 = 0 → falso (el residuo es 300)	1900	—
	Condición final: falso o falso = falso	1900	—
Salida	No se cumple la condición	1900	El año 1900 no es bisiesto

¿Es equivalente escribir las condiciones en otro orden?
Probé con 1900 y 2000 para comprobarlo.

Justificación: Si se pregunta primero año mod 4 = 0 y se responde "bisiesto" sin más revisiones, 1900 saldría bisiesto por error, porque es divisible entre 4. En cambio, si se pregunta primero año mod 400 = 0, luego año mod 100 = 0 y luego año mod 4 = 0, el resultado es correcto, porque cada excepción se revisa antes de la regla general.

1900	No bisiesto	Bisiesto (error)
2000	Bisiesto	Bisiesto

Problema 1.16. Número perfecto

Desarrolla un programa que lea un entero positivo y determine si es perfecto, es decir, si es igual a la suma de sus divisores propios (los menores que él).

1. Análisis del problema

Entrada: un número entero positivo n

Proceso: se recorren los posibles divisores d desde 1 hasta n-1; cada vez que n mod d = 0, d se suma a un acumulador.

Escaneado con CamScanner

Al terminar, se compara la suma con n.

Salida: un mensaje que indica si el número es perfecto o no.

Restricción: n debe ser mayor que 0.

2. Tabla de objetos

Objeto	Nombre	Valor	Tipo
Dato de entrada	n	Variable	Entero
Control	d	Variable	Entero
Trabajo	Suma	Variable	Entero
Dato de salida	Mensaje	Variable (pantalla)	Texto

3. Seudocódigo

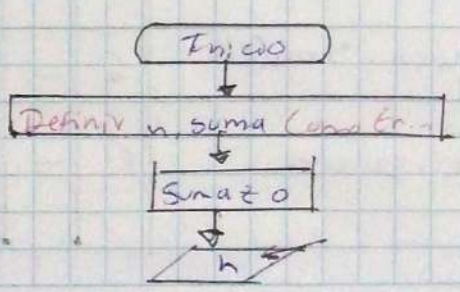
```
Algoritmo NumeroPerfecto
    Definir n, d, Suma Como Entero
    Suma ← 0;

    Escribir "Ingrese un número positivo: ";
    Leer n;

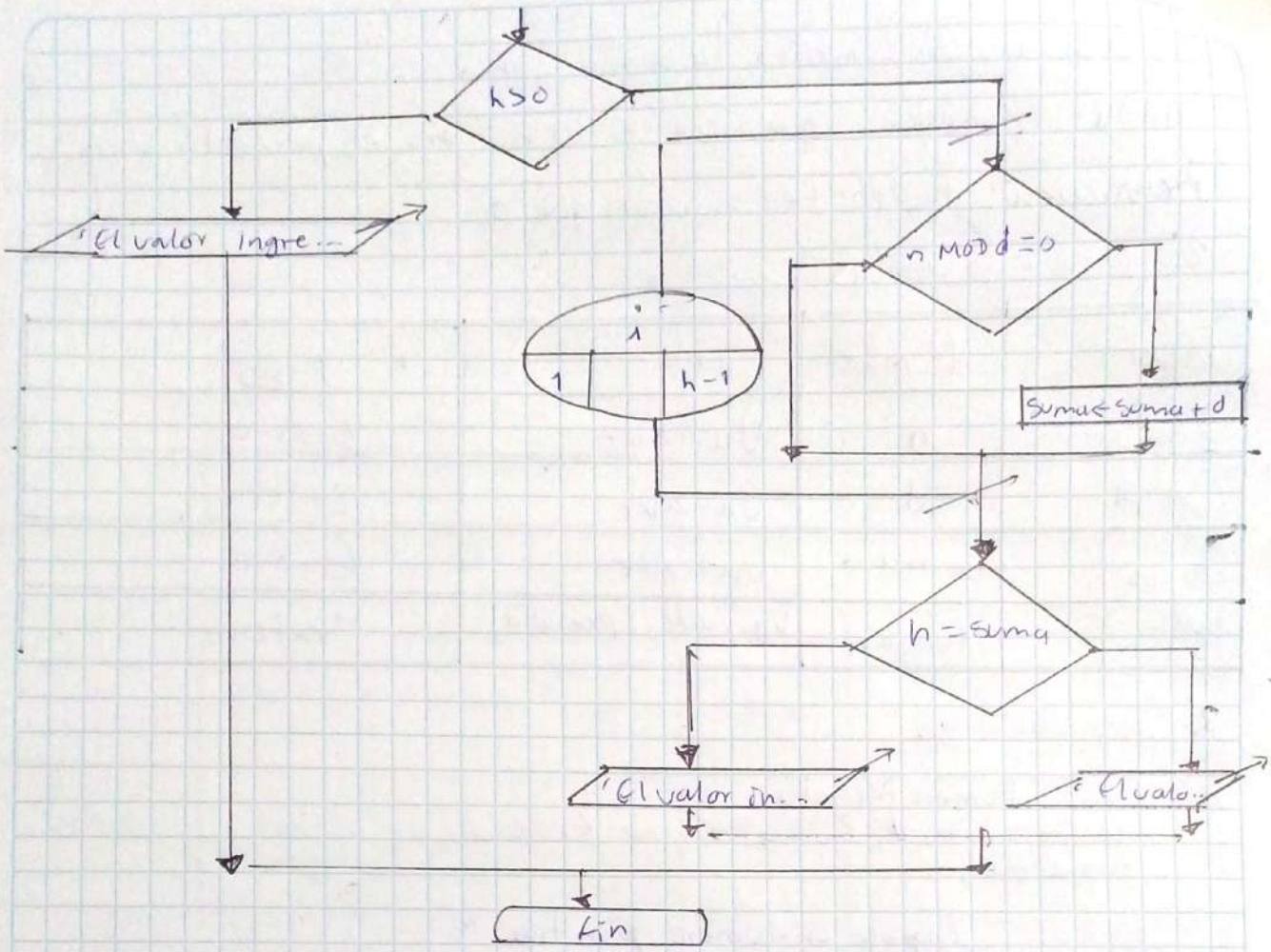
    Si n>0 Entonces
        Para d←1 Hasta n-1 Hacer
            Si n mod d = 0 Entonces
                Suma ← Suma + d;
            FinSi
        FinPara

        Si Suma = n Entonces
            Escribir "El número es perfecto";
        SiNo
            Escribir "El número no es perfecto";
        FinSi
    SiNO
        Escribir "El número debe ser positivo";
    FinSi
FinAlgoritmo
```

4. Algoritmo



Escaneado con CamScanner



5. Prueba de escritorio

Etapa	Descripción	n	d	n mod d	Suma	Pantalla
Entrada	El usuario escribe: 6	6	-	-	0	-
Proceso	Vuelta 1: 6 % 1 = 0 es divisor	6	1	0	1	-
	Vuelta 2: 6 % 2 = 0, es divisor	6	2	0	3	-
	Vuelta 3: 6 % 3 = 0, es divisor	6	3	0	6	-
	Vuelta 4: 6 % 4 = 2, no es divisor	6	4	2	6	-
	Vuelta 5: 6 % 5 = 1, no es divisor	6	5	1	6	-
	Comparación: suma(6) = n(6)	6	-	-	6	-
Salida	Son iguales	6	-	-	6	El número 6 es perfecto

Escaneado con CamScanner

¿Hasta qué valor es realmente necesario recorrer los divisores?

Ningún número mayor que la mitad de n puede ser divisor propio de n, porque el siguiente múltiplo de ese valor ya sería mayor que n (por ejemplo, el mayor divisor propio de 28 es 14, no 27). Por eso el ciclo puede ir de 1 hasta n/2 y se reduce a la mitad de las vueltas: con 28 pasaría de 27 a 14 vueltas.

Reducción aún mayor: como los divisores vienen en pares (d y n/d), basta recorrer hasta la raíz cuadrada de n y sumar ambos divisores del par cada vez que d divide a n. Con 28 bastarían 5 vueltas (hasta 5), en las que se encuentran los pares (1,28), (2,14) y (4,7), sin contar el propio n.

Escaneado con CamScanner

Problema 1.17. Conversión de decimal a binario

Desarrollar un programa que lea un entero positivo en base 10 y muestre su representación en binario.

1. Análisis del problema

Entrada: un número entero positivo (n).

Proceso: dividir sucesivamente el número entre 2 para obtener los residuos. Para resolver el orden inverso de los arreglos, se acumula el resultado multiplicando cada residuo por una potencia de 10 (1, 10, 100, ...) que se va incrementando en cada iteración.

Salida: la representación binaria del número.

Restricción: el valor de n debe ser un número positivo

2. Tabla de objetos

Objeto	Nombre	Valor	Tipo
Dato de entrada	n	Variable	Entero
De trabajo	cociente	Variable	Entero
De trabajo	residuo	Variable	Entero
De trabajo	factor	Variable	Entero
Dato de salida	binario	Variable	Entero

3. Seudocódigo

```
Algoritmo DecimalBinario
  Definir n, cociente, residuo, binario, factor Como Entero
  Escribir "Ingresa un entero positivo:"
  Leer n
  Si n>0 Entonces
    cociente ← n
    binario ← 0
    factor ← 1
    Mientras cociente > 0 Hacer
      residuo ← cociente MOD 2
      binario ← binario + (residuo * factor)
      cociente ← trunc(cociente/2)
      factor ← factor * 10
```

Escaneado con CamScanner

finMientras

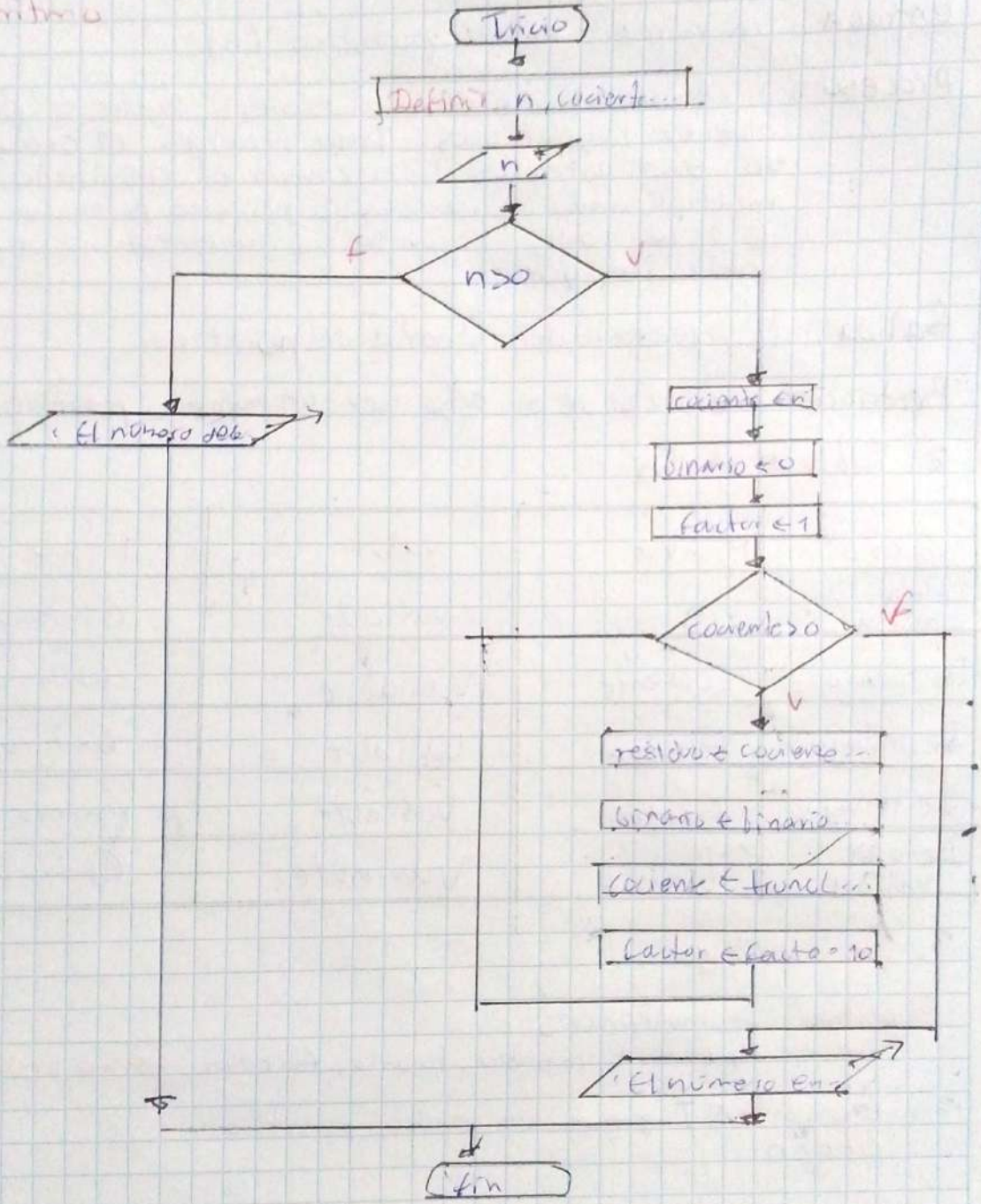
Escribir "El número en binario es: ", binario

SiNO

Escribir "El número debe ser un entero positivo".

finSi

finAlgoritmo



5. Prueba de escritorio

n (entrada)	cociente	residuo	factor	binario
13	13	–	1	0
13	6 (13/2)	1 (3 MOD 2)	10 (1*10)	0 + (1*1) = 1

Escaneado con CamScanner

13	3(6/2)	0(6 MOD 2)	100(10·10)	1+(1·100) = 101
13	1(3/2)	1(3 MOD 2)	1000(100·10)	1+(1·1000) = 1001
13	0(1/2)	1(1 MOD 2)	10000 (1000·10)	1001+(1·10000) = 11001

¿Cómo resolverías ese detalle sin usar arreglos?

Se resuelve utilizando una variable factor que comienza en 1 y se multiplica por 10 en cada iteración. Al multiplicar cada residuo por ese factor, los dígitos se posicionan automáticamente en orden inverso (unidades, decenas, centenas, etc), sin necesidad de usar arreglos.

Problema 1.14. Triangulo de asteriscos

Desarrolla un programa que lea un entero n≥0 y dibuje por pantalla un triángulo rectángulo de n filas, donde la fila i has- tiene i asteriscos.

1. Análisis del problema

Entrada: número entero (n).

Proceso: Utilizar un ciclo anidado, el ciclo externo controla las filas (desde 1 hasta n) y el ciclo interno controla la impresión de los asteriscos en cada fila (desde 1 hasta el número de la fila actual), haciendo un salto de linea al terminar cada fila.

Salida: el triangulo de asteriscos impreso en pantalla.

Restricción: el valor de entrada n debe ser un número entero mayor o igual a 1.

2. Tabla de objetos

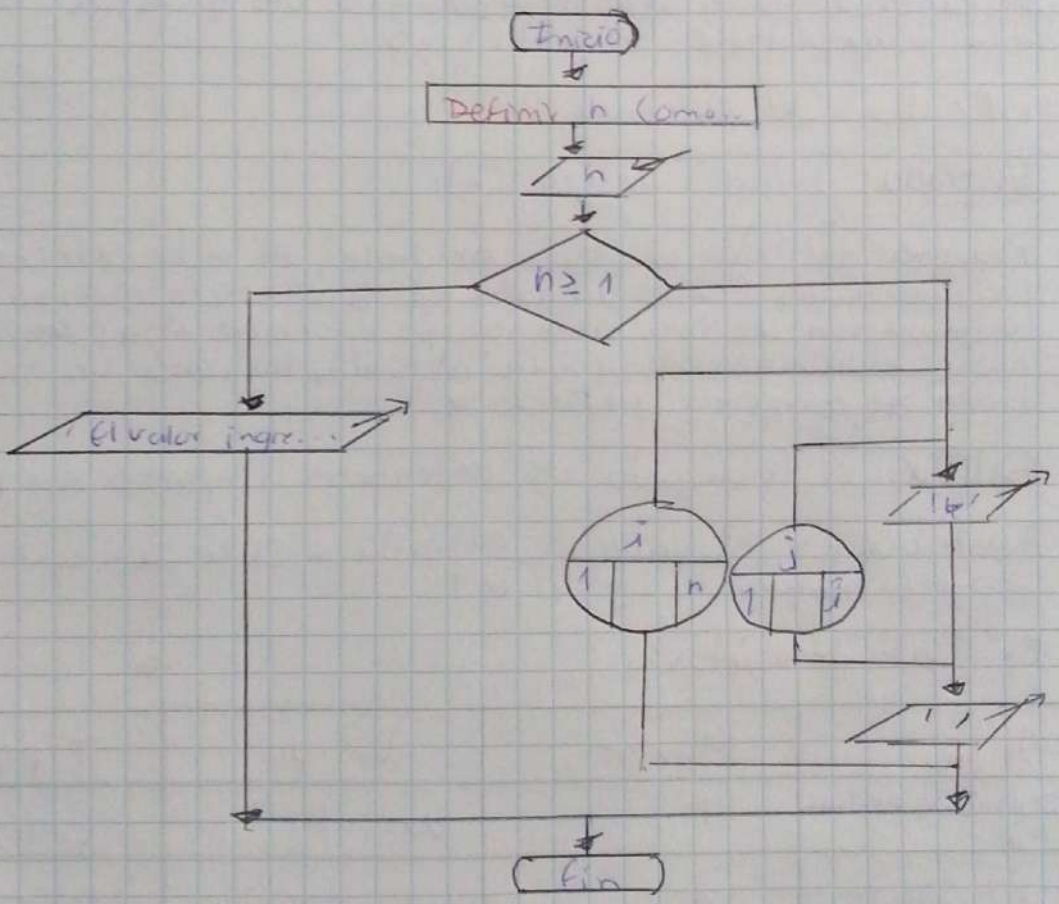
Objeto	Nombre	Valor	Tipo
Dato de entrada	n	Variable	Entero
De control	i	Variable	Entero
De control	j	Variable	Entero
Dato de salida	triángulo	Variable	Entero

Escaneado con CamScanner

3. Seudocodigo

```
Algoritmo TrianguloAsterisco
    Definir n, j, i Como Entero
    Escribir "Ingresa un entero n: ";
    Leer n
    Si n >= 1 Entonces
        Para i ← 1 Hasta n Hacer
            Para j ← 1 Hasta i Hacer
                Escribir Sin Saltar "* "
            FinPara
            Escribir " "
        FinPara
    SiNo
        Escribir "El número debe ser mayor o igual a 1".
    FinSi
FinAlgoritmo
```

4. Algoritmo



Escaneado con CamScanner

5. Prueba de escritorio

Entrada	Proceso	Salida
n = 3	–	–
3 ≥ 1	–	–
n = 3	j toma el valor de 1	*
n = 3	j toma valores de 1 a 2	* *
n = 3	j toma valores de 1 a 3	* * *

¿Qué parte de tu algoritmo cambiaría y qué parte se mantendría?

Se mantendrían los ciclos anidados de filas, pero cambiaría el proceso interno para imprimir los espacios previos y ajustar los asteriscos a una secuencia impar.

Problema 1.19. Segundo mayor de una secuencia

Desarrolla un programa que lea números enteros positivos hasta que se introduzca un 0, y muestre el mayor y el segundo mayor de los valores ingresados. Si hay menos de dos valores, debe informarlo.

1. Análisis del problema

Entrada: Secuencia de números enteros

Proceso: actualizar las variables mayor y segundo comparando cada número ingresado.

Salida: el mayor y el segundo mayor, o un mensaje si hay menos de dos valores.

Restricción: se requieren al menos dos valores válidos distintos de cero.

2. Tabla de objetos

Objeto	Nombre	Valor	Tipo
Entrada	n	variable	Entero
Trabajo	mayor	0	Entero
Trabajo	segundo	0	Entero

Escaneado con CamScanner

Salida	resultado	Pantalla	Texto

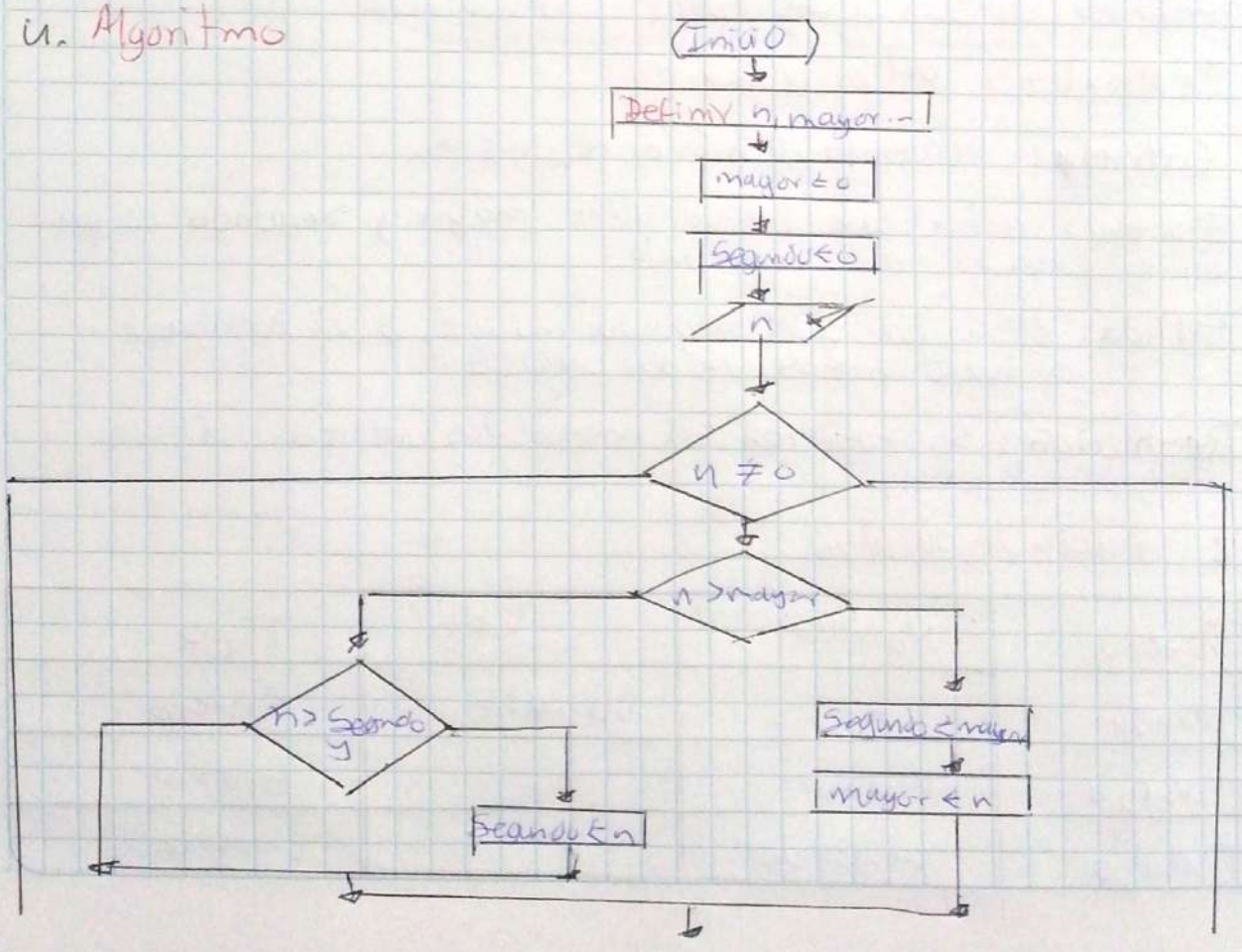
3. Seudocódigo

```
Algoritmo SegundoMayor
    Definir n, mayor, segundo Como Entero
    mayor ← 0
    segundo ← 0
    Escribir "Ingrese un número: "
    Leer n,

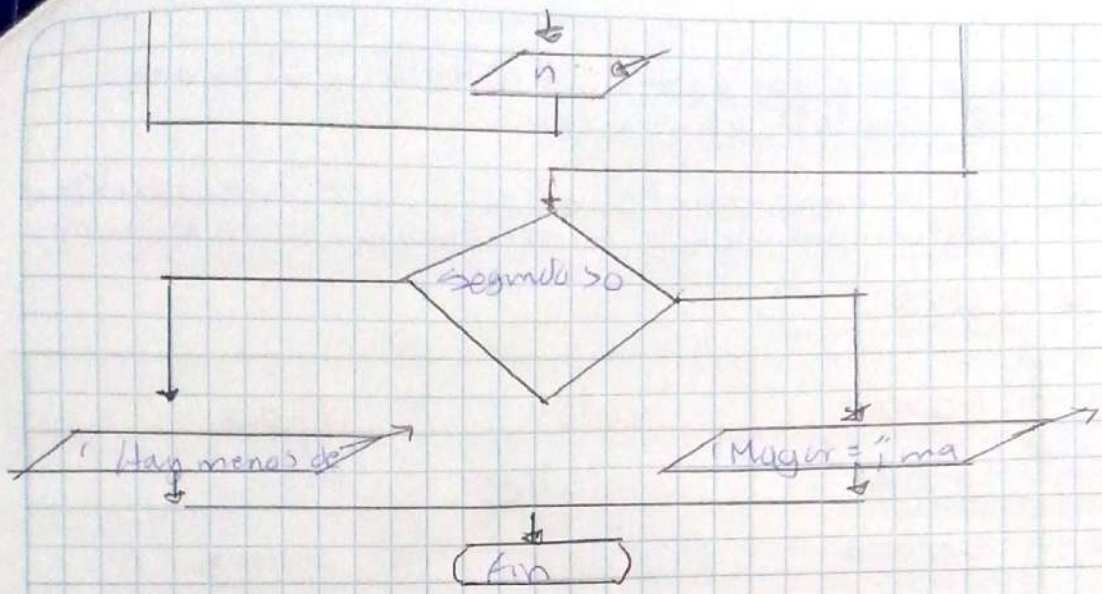
    Mientras n≠0 Hacer
        Si n>mayor entonces
            segundo ← mayor
            mayor ← n
        SiNo
            Si n>segundo Y n<mayor Entonces
                segundo ← n
            FinSi
        FinSi
        Leer n

    Si segundo >0 Entonces
        Escribir "Mayor = ", mayor, "; Segundo mayor = ", segundo
    SiNo
        Escribir "Hay menos de dos valores"
    FinSi
FinAlgoritmo
```

4. Algoritmo



Escaneado con CamScanner



5. Prueba de escritorio

Paso	Entrada	Proceso	Salida
lectura 1	7	mayor = 7, segundo = 0	—
lectura 2	3	mayor = 7, segundo = 3	—
lectura 3	9	mayor = 9, segundo = 7	—
lectura 4	5	mayor = 9, segundo = 7	—
lectura 5	0	Fin del ciclo (0 + 0) Falso	Mayor = 9, Segundo = 7

¿Qué debería pasar si el valor mayor se repite (por ejemplo, 9, 9, 5)?

En la condición actual (n < mayor) descarta los números iguales. Si se quiere permitir valores repetidos, el segundo (5) debería evaluar si n >= segundo y n < mayor asegurando que tome el valor repetido en lugar de ignorarlo.

Programa 1.20. Adivina el número

Desarrolla un programa que "piense" un número 1 y 100 (puede ser fijo para la práctica) y pida al usuario adivinarlo. Después de cada intento debe indicar "más alto", "más bajo" o "¡acertaste!", y al final mostrar cuántos intentos se usaron.

1. Análisis del problema

Entrada: número ingresado por el usuario (intento).

Proceso: comparar el número ingresado con el número secreto, incrementar el contador de intentos e indicar si el número buscado es mayor o menor.

Escaneado con CamScanner

Salida: mensaje de pista tras cada intento y el número total de intentes al acertar.

Restricción: el número secreto es fijo y los intentos deben ser comparados relativamente hasta acertar.

2. Tabla de objetos

Objetos	Nombre	Valor	Tipo
Dato de entrada	intento	Variable	Entero
De trabajo	secreto	Fijo	Entero
De trabajo	contador	variable	Entero
Dato de salida	resultado	Pantalla	Texto

3. Seudocódigo

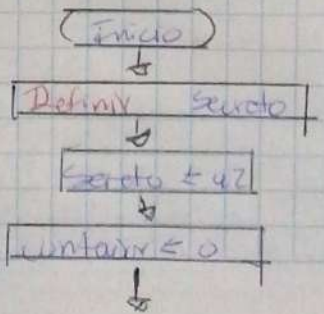
```
Algoritmo AdivinaElNumero
    Definir secreto, intento, contador Como Entero
    secreto ← 42
    contador ← 0

    Repetir
        Escribir "Ingrese tu número (1-100):"
        Leer intento
        contador ← contador + 1

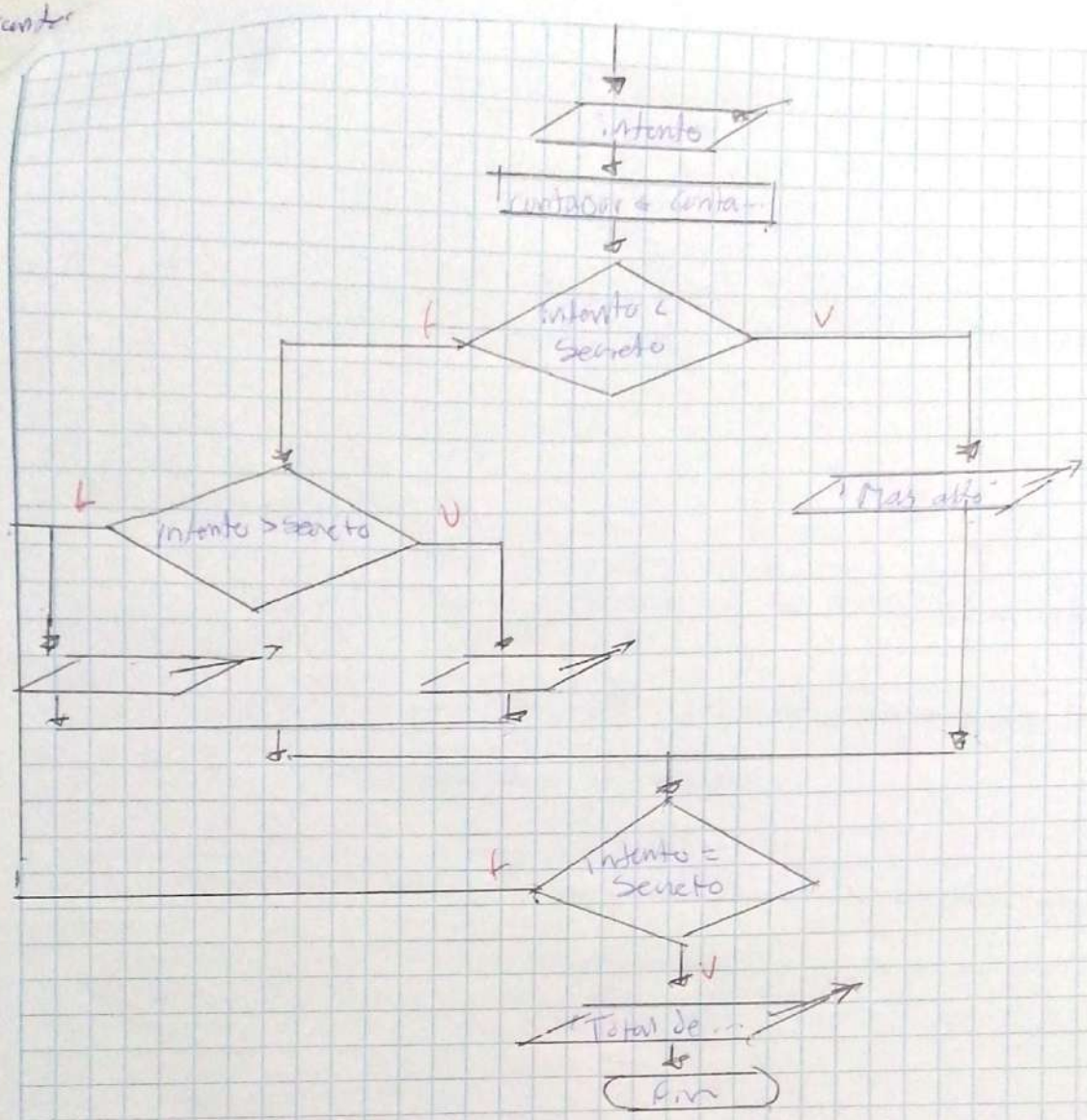
        Si intento < secreto Entonces
            Escribir "Más alto"
        SiNo
            Si intento > secreto Entonces
                Escribir "Más bajo"
            SiNo
                Escribir "¡Acertaste!";
            FinSi
        FinSi
    Hasta Que intento = secreto

    Escribir "Total de intentes: ", contador
FinAlgoritmo
```

4. Algoritmo



Escaneado con CamScanner



5. Prueba de escritorio

Paso	Entrada	Proceso	Salida
Intento 1	50	50 > 42 (contador = 1)	Más bajo
Intento 2	25	25 < 42 (contador = 2)	Más alto
Intento 3	42	42 = 42 (contador = 3)	¡Acertaste! Total de intentos: 3

¿Cuál es el máximo de intentos que necesita para el rango 1-100?

Se necesita como máximo 7 intentos. Al utilizar búsqueda binaria, el espacio de búsqueda se divide a la mitad en cada paso (100 → 50 → 25 → 12 → 6 → 3 → 1). Matemáticamente, el logaritmo base 2 de 100 es ≈ 6.64, y al redondear hacia arriba se obtienen 7 intentos.

Escaneado con CamScanner